

# LOG430 Labo 03 — Activité 4 : endpoint GraphQL

Oct 3, 2026 · @ilyes

## Objectif et mise en place

L'activité 4 consiste à interroger l'endpoint `POST /stocks/graphql-query` avec GraphQL, où le client choisit lui-même les champs qu'il veut recevoir.

Avant la requête, nous avons préparé l'environnement en quatre étapes :

1. Lancement des conteneurs `store_manager`, `mysql` et `redis` avec `docker compose up -d` sur le réseau `labo03-network`.
2. Création de la table `stocks` dans `labo03_db`, avec une clé étrangère vers `products`, conformément au modèle `.mwb`.
3. Insertion de stocks de test pour les produits 1 à 4.
4. Appel de `POST /stocks` avec `{"product_id": 1, "quantity": 1000}` pour écrire la quantité dans Redis. Cette étape est nécessaire parce que le resolver GraphQL lit Redis et non MySQL.

## Question 3 : résultat de la requête suggérée

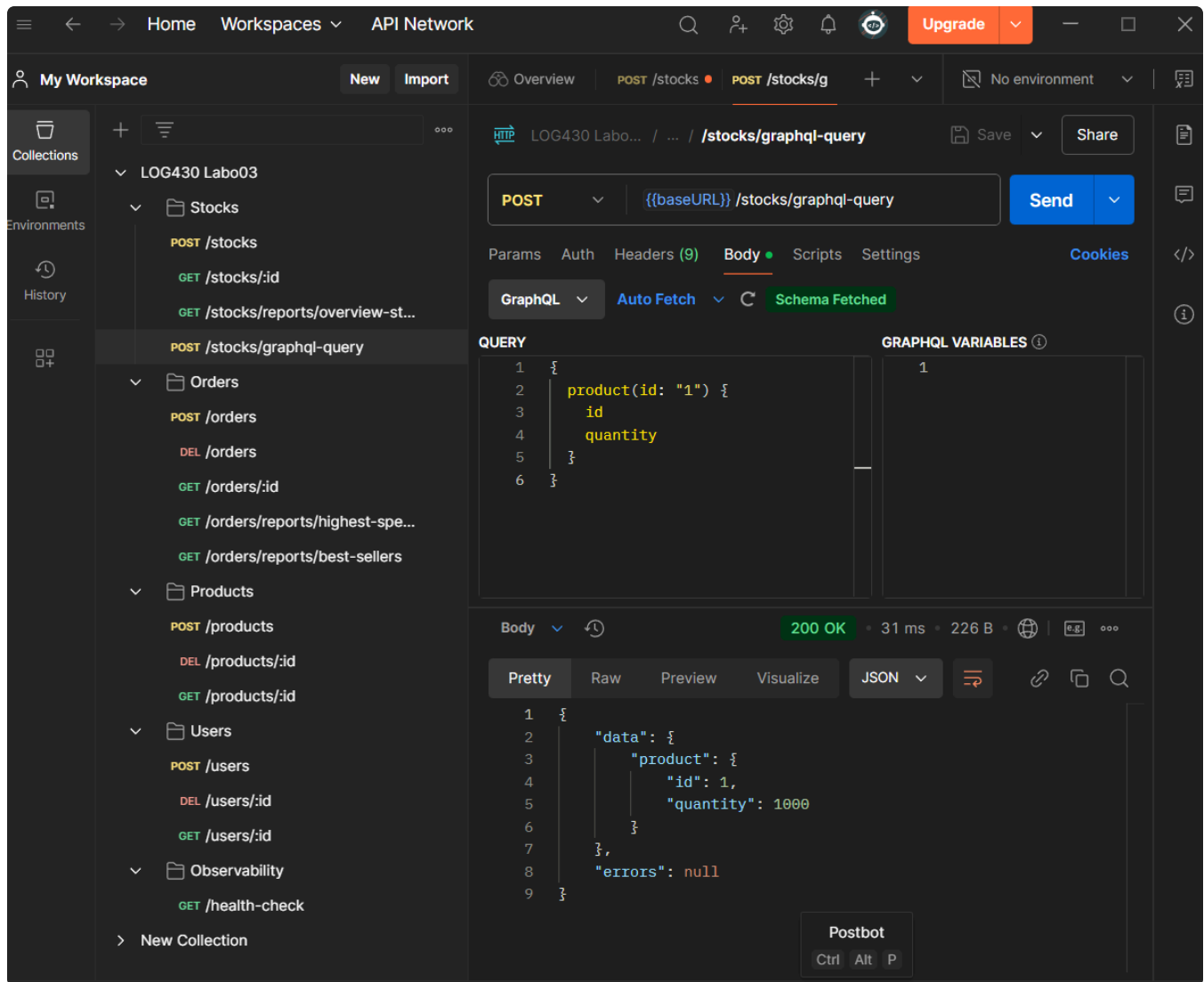
La requête suggérée renvoie le produit 1 avec seulement les deux champs demandés, `id` et `quantity`, et un statut `200 OK`.

Requête envoyée dans Postman (body en mode GraphQL) :

```
{
  product(id: "1") {
    id
    quantity
  }
}
```

Réponse obtenue :

```
{
  "data": {
    "product": {
      "id": 1,
      "quantity": 1000
    }
  },
  "errors": null
}
```



Côté serveur, Flask passe la requête au schéma Graphene avec `Schema(query=Query).execute(...)`. Le schéma appelle `resolve_product`, qui lit le hash Redis `stock:1` :

```
product_data = redis_client.hgetall(f"stock:{id}")
```

GraphQL ne renvoie ensuite que les champs demandés. Avec une route REST classique, c'est le serveur qui fixe la structure de la réponse.

## Observations complémentaires

Trois requêtes supplémentaires révèlent les limites de l'implémentation actuelle.

| Requête                                                | Résultat                                                                                   | Cause                                                                                                                                                              |
|--------------------------------------------------------|--------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>product(id: "1") { id<br/>name quantity }</code> | <code>"name": "Product 1"</code> au lieu de « Laptop ABC »                                 | Le resolver génère le nom ( <code>name=f"Product {id}"</code> ), car Redis ne stocke que <code>quantity</code>                                                     |
| <code>product(id: "1") { id<br/>sku }</code>           | <code>"data": null</code> , erreur <code>Cannot query field 'sku' on type 'Product'</code> | <code>sku</code> n'est pas déclaré dans le type GraphQL <code>Product</code> ( <code>schemas/product.py</code> ); la requête est rejetée avant l'appel du resolver |
| <code>product(id: "3") { id<br/>quantity }</code>      | <code>"product": null</code> , aucune erreur                                               | Aucune clé <code>stock:3</code> dans Redis, alors que le stock existe dans MySQL                                                                                   |

Requête avec `name` :

The screenshot shows the Postman API client interface. The left sidebar displays a collection named "LOG430 Labo03" with several endpoints. The main panel shows a POST request to the endpoint `POST /stocks/graphql-query` with a GraphQL query. The query is:

```
1 {
2   product(id: "1") {
3     id
4     name
5     quantity
6   }
7 }
```

The response is a 200 OK status with a response time of 16 ms and a body size of 245 B. The response body is displayed in JSON format:

```
1 {
2   "data": {
3     "product": {
4       "id": 1,
5       "name": "Product 1",
6       "quantity": 1000
7     }
8   },
9   "errors": null
10 }
```

The interface also shows a "GraphQL VARIABLES" section with a single variable `1`. The bottom status bar indicates "Postbot" and "Runner" are active.

Requête avec `sku` :

The screenshot displays the API Network client interface. On the left, the 'My Workspace' sidebar shows a collection named 'LOG430 Labo03' with several endpoints. The 'Stocks' collection is expanded, showing endpoints like 'POST /stocks', 'GET /stocks/id', and 'POST /stocks/graphql-query'. The 'Orders' collection is also expanded, showing endpoints like 'POST /orders', 'DEL /orders', 'GET /orders/id', and 'GET /orders/reports/highest-spe...'. The 'Products' collection is expanded, showing endpoints like 'POST /products', 'DEL /products/id', and 'GET /products/id'. The 'Users' collection is expanded, showing endpoints like 'POST /users', 'DEL /users/id', and 'GET /users/id'. The 'Observability' collection is expanded, showing the endpoint 'GET /health-check'. The 'New Collection' button is visible at the bottom of the sidebar.

The main panel shows the 'POST /stocks/graphql-query' endpoint selected. The 'Body' tab is active, displaying the GraphQL query:

```
1 {
2   product(id: "1") {
3     id
4     sku
5   }
6 }
```

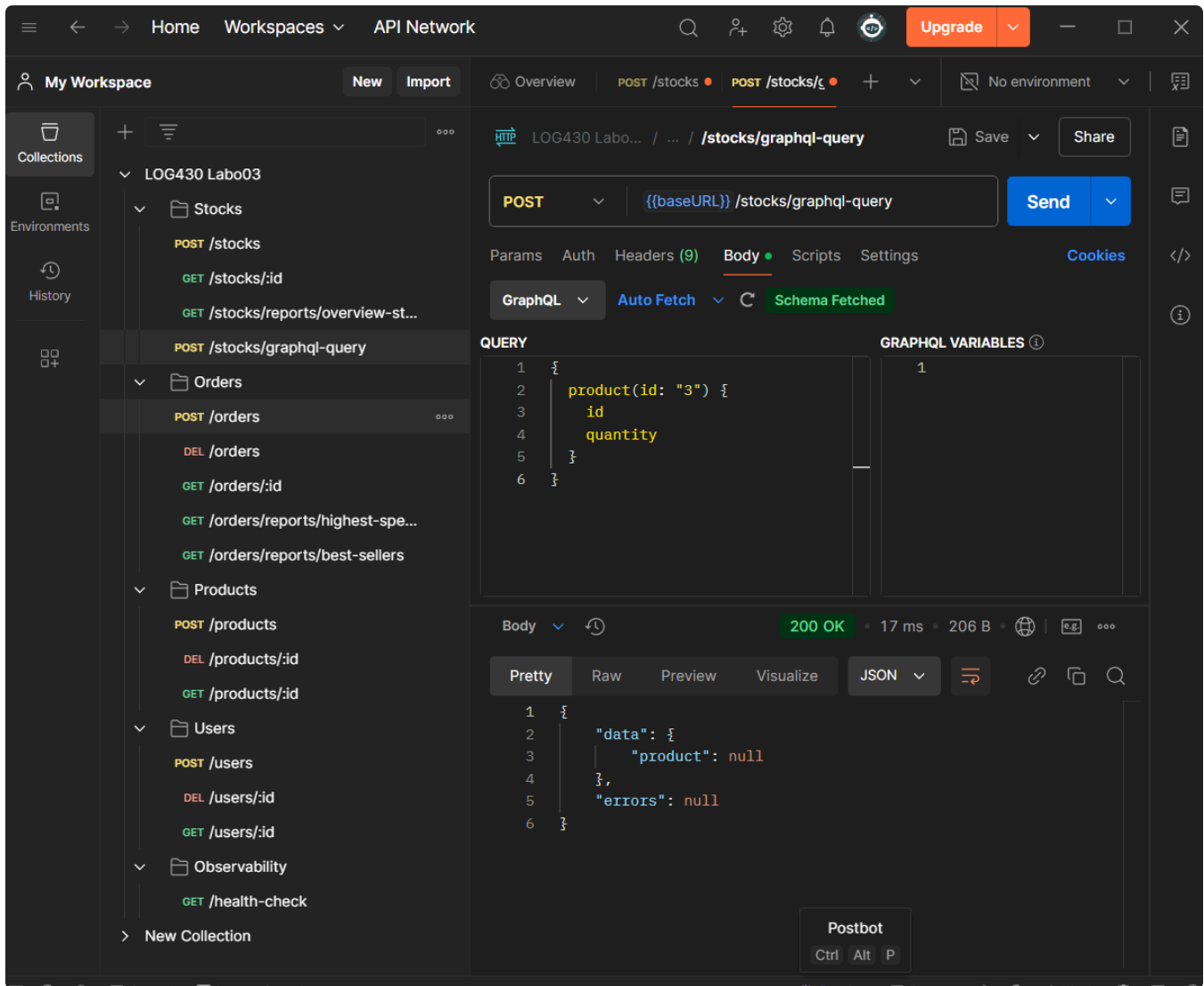
The 'GRAPHQL VARIABLES' section is empty.

The response is a 200 OK status with a response time of 17 ms and a body size of 306 B. The response is displayed in the 'Body' tab, showing the JSON response:

```
1 {
2   "data": null,
3   "errors": [
4     "Cannot query field 'sku' on type 'Product'.
5     \n\nGraphQL request:4:5\n3 |   id\n4 |   sku\n5 |   }\n6 | }
```

The 'Postbot' button is visible at the bottom right of the interface.

Produit 3, absent de Redis :



Dans le deuxième cas, le statut HTTP reste **200 OK** : GraphQL signale les erreurs dans le champ **errors** du corps de la réponse, et non par le code HTTP.

## Conclusion

L'endpoint GraphQL fonctionne et laisse le client choisir ses champs. En revanche, il n'expose aujourd'hui que **id** et **quantity** de manière fiable, et dépend entièrement du cache Redis. L'activité 5 corrige les deux premières limites : elle ajoute **name**, **sku** et **price** au type **Product** et au resolver, puis enrichit **update\_stock\_redis** pour stocker ces informations dans Redis.